# Julia → GLSL transpiler overview

SZŰCS GERGELY

# Usage (Rust GPU)

```rust
use glam::{Vec3, Vec4, vec2, vec3};

#[spirv(fragment)]
pub fn main(
    #[spirv(frag_coord)] in_frag_coord: &Vec4,
    #[spirv(push_constant)] constants: &ShaderConstants,
    output: &mut Vec4,
) {
    let frag_coord = vec2(in_frag_coord.x, in_frag_coord.y);
    let mut uv = (frag_coord - 0.5 * vec2(constants.width as f32, constants.height as f32))
        / constants.height as f32;
    uv.y = -uv.y;

    let eye_pos = vec3(0.0, 0.0997, 0.2);
    let sun_pos = vec3(0.0, 75.0, -1000.0);
    let dir = get_ray_dir(uv, eye_pos, sun_pos);

    // evaluate Preetham sky model
    let color = sky(dir, sun_pos);

    *output = tonemap(color).extend(1.0)
}
```

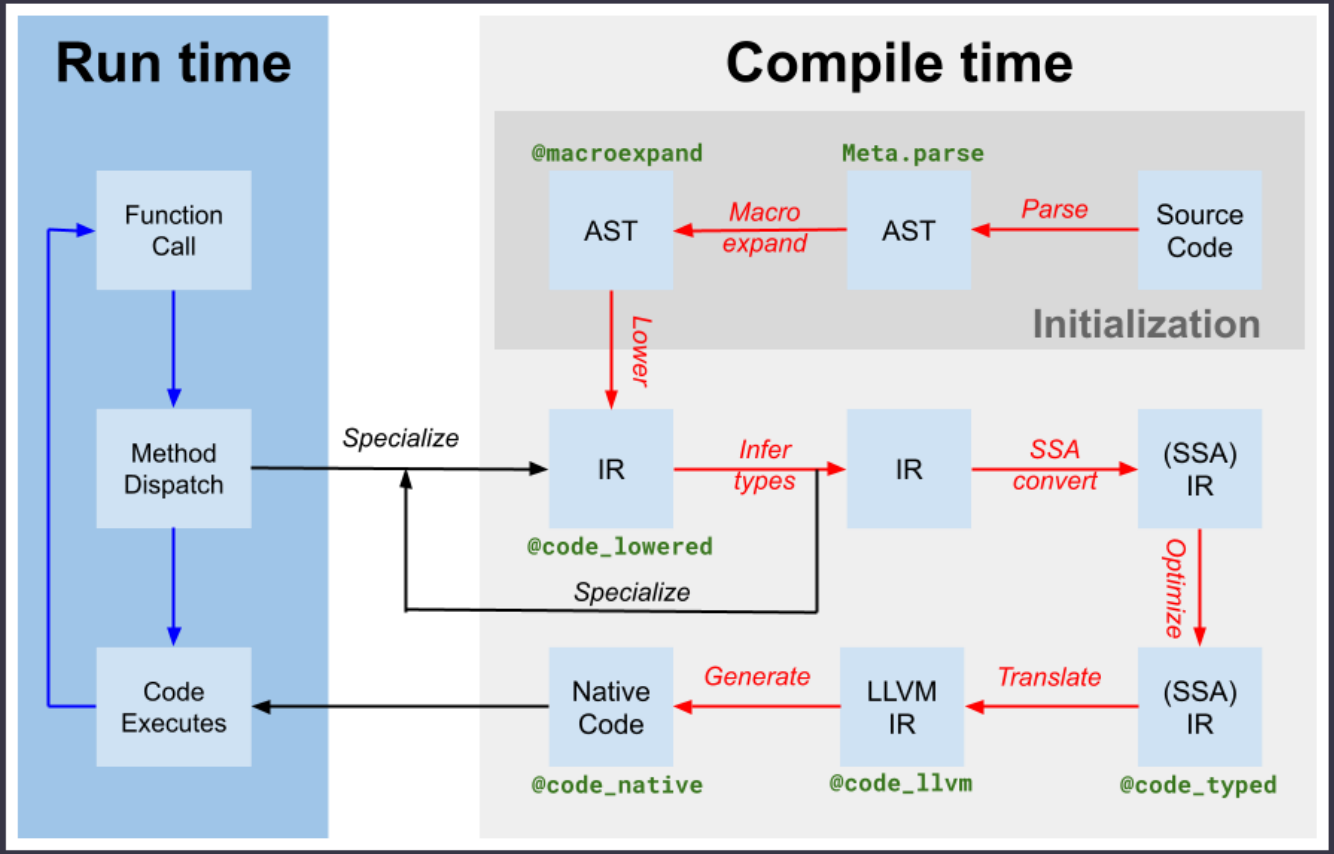
# Usage (with macros)

```
29
30  @frag function main(@out(fragColor::Ref{Vec4}))
31      fragColor[] = Vec4(1.0, 0.0, 0.0, 1.0)
32  end
33
```

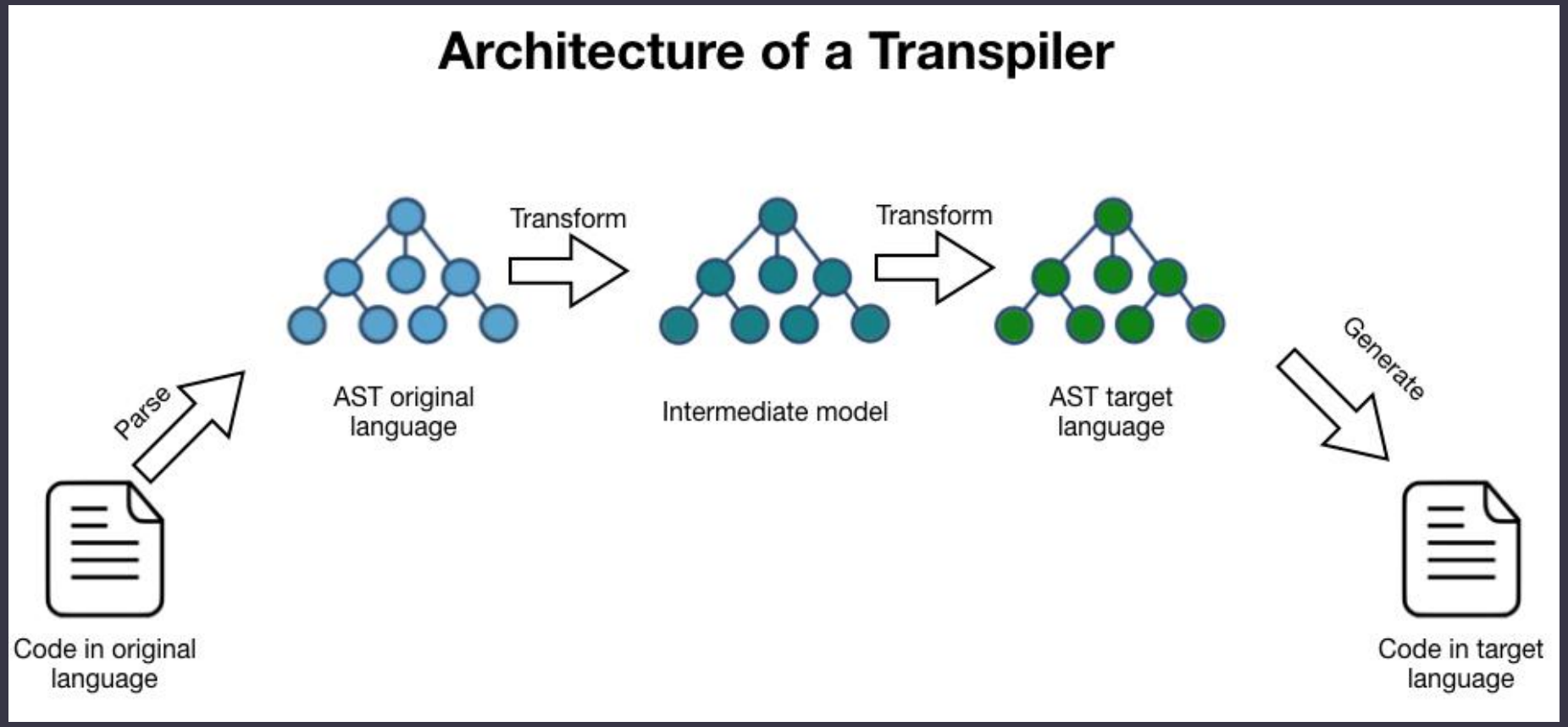
# Usage (with macros)

```
35
36  @frag function main(
37      @uniform(time::Float32),
38      @out(fragColor::Ref{Vec4})
39  )
40      fade = 0.5 * (1.0 + sin(time))
41      fragColor[] = Vec4(fade, 0.0, 0.0, 1.0)
42  end
43
```

# AST access in Julia



# AST transformation



# AST transformation

```
julia> expr = :(some_val = 0.0)
:(some_val = 0.0)

julia> dump(expr)
Expr
  head: Symbol =
  args: Array{Any}((2,))
    1: Symbol some_val
    2: Float64 0.0
```

```
Sequence
  move second child to first child ( temp float)
    'some_val' ( temp float)
    Constant:
      0.000000
```

```julia
julia> expr2 = quote
            for i = 1:5
                some_val += i
            end
        end
```

```julia
julia> dump(expr2)
Expr
  head: Symbol block
  args: Array{Any}((2,))
    1: LineNumberNode
      line: Int64 2
      file: Symbol REPL[14]
    2: Expr
      head: Symbol for
      args: Array{Any}((2,))
        1: Expr
          head: Symbol =
          args: Array{Any}((2,))
            1: Symbol i
            2: Expr
              head: Symbol call
              args: Array{Any}((3,))
                1: Symbol :
                2: Int64 1
                3: Int64 5
        2: Expr
          head: Symbol block
          args: Array{Any}((3,))
            1: LineNumberNode
              line: Int64 3
              file: Symbol REPL[14]
            2: Expr
              head: Symbol +=
              args: Array{Any}((2,))
                1: Symbol some_val
                2: Symbol i
            3: LineNumberNode
              line: Int64 4
              file: Symbol REPL[14]
```

```java
for (int i = 1; i <= 5; i++) {
    some_val += i;
}
```

```
Sequence
  Sequence
    move second child to first child ( temp int)
      'i' ( temp int)
      Constant:
        1 (const int)
  Loop with condition tested first
    Loop Condition
    Compare Less Than or Equal ( temp bool)
      'i' ( temp int)
      Constant:
        5 (const int)
    Loop Body
    Sequence
      add second child into first child ( temp float)
        'some_val' ( temp float)
        Convert int to float ( temp float)
          'i' ( temp int)
    Loop Terminal Expression
    Post-Increment ( temp int)
      'i' ( temp int)
```

# Dynamic → static typing

```
55
56  x = 3.0
57  # perform calculations with x as a float
58  # ...
59
60  x = Vec4(1.0, 3.0, 0.0, 1.0)
61  # perform calculations with x as a vec4
62  # ...
63
```

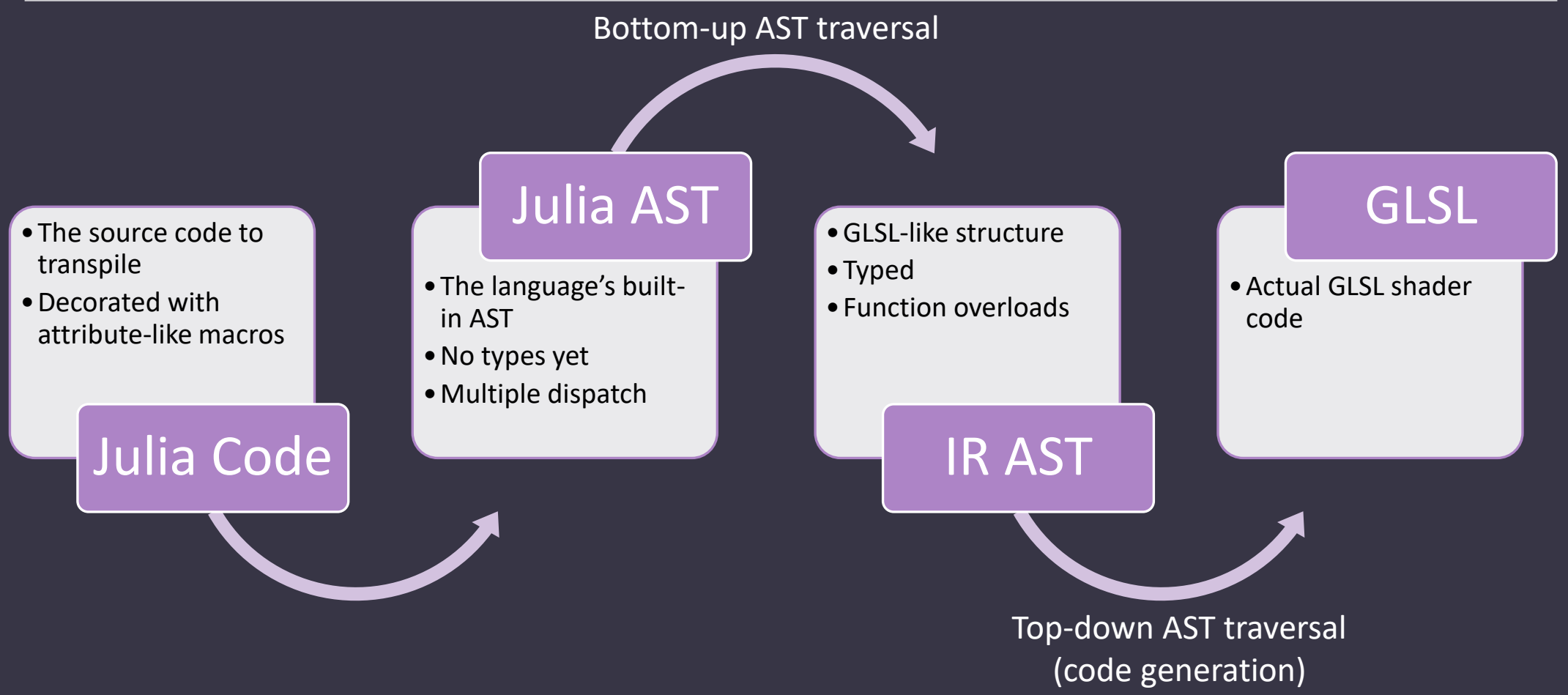
```
2
3   float x1 = 3.0;
4   // perform calculations with x1
5   // ...
6
7   // x1 isn't used again from this point
8   vec4 x2 = vec4(1.0, 3.0, 0.0, 1.0);
9   // perform calculations with x2
10
```

# AST traversal

```
35
36  @frag function main(
37      @uniform(time::Float32),
38      @out(fragColor::Ref{Vec4})
39  )
40      fade = 0.5 * (1.0 + sin(time))
41      fragColor[] = Vec4(fade, 0.0, 0.0, 1.0)
42  end
43
```

```
:block
├─ LineNumberNode
│  └─ ...
├─ :=
│  ├─ :fade
│  └─ :call
│     ├─ :*
│     ├─ Float32 0.5
│     └─ :call
│        ├─ :+
│        ├─ Float32 1.0
│        └─ :call
│           ├─ :sin
│           └─ :time
├─ LineNumberNode
│  └─ ...
└─ :=
   ├─ :ref
   │  └─ :fragColorOut
   └─ :call
      ├─ :Vec4
      ├─ :fade
      ├─ Float32 0.0
      ├─ Float32 0.0
      └─ Float32 1.0
```

# Overview


Bottom-up AST traversal
Julia Code
• The source code to transpile
• Decorated with attribute-like macros
Julia AST
• The language's built-in AST
• No types yet
• Multiple dispatch
IR AST
• GLSL-like structure
• Typed
• Function overloads
GLSL
• Actual GLSL shader code
Top-down AST traversal
(code generation)